

SKILL 2

The screenshot shows the MySQL Workbench interface. The main editor displays a series of SQL queries: dropping the 'orders' table if it exists, creating it with columns 'ord_no' (INT PRIMARY KEY), 'purch_amt' (DECIMAL(10,2)), 'ord_date' (DATE), 'customer_id' (INT), and 'salesman_id' (INT), and then inserting 12 rows of data. The 'Output' tab at the bottom shows the execution log, indicating that the table was successfully created and 12 rows were inserted. The 'Schemas' pane on the left shows the database structure, including the 'student' database and its tables.

```
3 DROP TABLE IF EXISTS orders;
4 CREATE TABLE orders (
5   ord_no INT PRIMARY KEY,
6   purch_amt DECIMAL(10,2),
7   ord_date DATE,
8   customer_id INT,
9   salesman_id INT
10 );
11 INSERT INTO orders VALUES
12 (70001, 150.50, '2012-10-05', 3005, 5002),
13 (70009, 270.65, '2012-09-10', 3001, 5005),
14 (70002, 65.26, '2012-10-05', 3002, 5001),
15 (70004, 110.50, '2012-08-17', 3009, 5003),
16 (70007, 948.50, '2012-09-10', 3005, 5002),
17 (70005, 2400.60, '2012-07-27', 3007, 5001),
18 (70008, 5760.00, '2012-09-10', 3002, 5001),
19 (70010, 1983.43, '2012-10-10', 3004, 5006),
20 (70003, 2480.40, '2012-10-10', 3009, 5003),
21 (70012, 250.45, '2012-06-27', 3008, 5002),
22 (70011, 75.29, '2012-08-17', 3003, 5007),
23 (70013, 3045.60, '2012-04-25', 3002, 5001);
```

#	Time	Action	Message	Duration / Fetch
36	19:43:06	SELECT subject, COUNT(*) AS num_students FROM student_marks GROUP BY subject HAVING COUNT(*) > 1	1 row(s) returned	0.000 sec / 0.000 sec
37	19:49:46	CREATE DATABASE IF NOT EXISTS orders_db	1 row(s) affected	0.015 sec
38	19:50:02	USE orders_db	0 row(s) affected	0.000 sec
39	19:50:12	DROP TABLE IF EXISTS orders	0 row(s) affected, 1 warning(s): 1051 Unknown table 'orders_db.orders'	0.016 sec
40	19:50:29	CREATE TABLE orders (ord_no INT PRIMARY KEY, purch_amt DECIMAL(10,2), ord_date DATE, customer_id INT, salesman_id INT)	0 row(s) affected	0.031 sec
41	19:50:44	INSERT INTO orders VALUES (70001, 150.50, '2012-10-05', 3005, 5002), (70009, 270.65, '2012-09-10', 3001, 5005), (70002, 65.26, '2012-10-05', 3002, 5001), (70004, 110.50, '2012-08-17', 3009, 5003), (70007, 948.50, '2012-09-10', 3005, 5002), (70005, 2400.60, '2012-07-27', 3007, 5001), (70008, 5760.00, '2012-09-10', 3002, 5001), (70010, 1983.43, '2012-10-10', 3004, 5006), (70003, 2480.40, '2012-10-10', 3009, 5003), (70012, 250.45, '2012-06-27', 3008, 5002), (70011, 75.29, '2012-08-17', 3003, 5007), (70013, 3045.60, '2012-04-25', 3002, 5001);	12 row(s) affected Records: 12 Duplicates: 0 Warnings: 0	0.000 sec

```
24 SELECT *
25 FROM orders
26 WHERE purch_amt > 2000;
```

Result Grid					
Filter Rows:					
	ord_no	purch_amt	ord_date	customer_id	salesman_id
	70003	2480.40	2012-10-10	3009	5003
	70005	2400.60	2012-07-27	3007	5001
	70008	5760.00	2012-09-10	3002	5001
	70013	3045.60	2012-04-25	3002	5001
	NULL	NULL	NULL	NULL	NULL

```

27 • SELECT *
28 FROM orders
29 WHERE ord_date = '2012-09-10';

```

Result Grid					
Filter Rows:					
Edit: 					
	ord_no	purch_amt	ord_date	customer_id	salesman_id
▶	70007	948.50	2012-09-10	3005	5002
	70008	5760.00	2012-09-10	3002	5001
	70009	270.65	2012-09-10	3001	5005
•	NULL	NULL	NULL	NULL	NULL

```

30 • SELECT *
31 FROM orders
32 WHERE salesman_id = 5001;

```

Result Grid					
Filter Rows:					
Edit: 					
	ord_no	purch_amt	ord_date	customer_id	salesman_id
▶	70002	65.26	2012-10-05	3002	5001
	70005	2400.60	2012-07-27	3007	5001
	70008	5760.00	2012-09-10	3002	5001
	70013	3045.60	2012-04-25	3002	5001
•	NULL	NULL	NULL	NULL	NULL

```

33 • SELECT *
34 FROM orders
35 ORDER BY purch_amt DESC;

```

Result Grid					
Filter Rows:					
Edit: 					
	ord_no	purch_amt	ord_date	customer_id	salesman_id
▶	70008	5760.00	2012-09-10	3002	5001
	70013	3045.60	2012-04-25	3002	5001
	70003	2480.40	2012-10-10	3009	5003
	70005	2400.60	2012-07-27	3007	5001
	70010	1983.43	2012-10-10	3004	5006
	70007	948.50	2012-09-10	3005	5002
	70009	270.65	2012-09-10	3001	5005
	70012	250.45	2012-06-27	3008	5002
	70001	150.50	2012-10-05	3005	5002
	70004	110.50	2012-08-17	3009	5003
	70011	75.29	2012-08-17	3003	5007

```

36 • SELECT *
37 FROM orders
38 ORDER BY ord_date;

```

Result Grid					
Filter Rows:					
Edit: 					
	ord_no	purch_amt	ord_date	customer_id	salesman_id
▶	70013	3045.60	2012-04-25	3002	5001
	70012	250.45	2012-06-27	3008	5002
	70005	2400.60	2012-07-27	3007	5001
	70004	110.50	2012-08-17	3009	5003
	70011	75.29	2012-08-17	3003	5007
	70007	948.50	2012-09-10	3005	5002
	70008	5760.00	2012-09-10	3002	5001
	70009	270.65	2012-09-10	3001	5005
	70001	150.50	2012-10-05	3005	5002
	70002	65.26	2012-10-05	3002	5001
	70003	2480.40	2012-10-10	3009	5003

```
39 • SELECT SUM(purch_amt) AS total_revenue
40 FROM orders;
```

<

Result Grid



Filter Rows:

Export:



	total_revenue
▶	17541.18

```
41 • SELECT AVG(purch_amt) AS average_order
42 FROM orders;
```

<

Result Grid



Filter Rows:

Export:



Wrap Cell

	average_order
▶	1461.765000

```
43 • SELECT MAX(purch_amt) AS highest_order
44 FROM orders;
```

<

Result Grid



Filter Rows:

Export:



Wrap Cell C

	highest_order
▶	5760.00

```
45 • SELECT MIN(purch_amt) AS lowest_order
46 FROM orders;
```

<

Result Grid



Filter Rows:

Export:



Wrap Cell Content:

	lowest_order
▶	65.26

```
47 • SELECT COUNT(*) AS total_orders
48 FROM orders;
```

<

Result Grid



Filter Rows:

Export:

	total_orders
▶	12

```
48 FROM orders;
49 • SELECT salesman_id,
50        SUM(purch_amt) AS total_sales
51 FROM orders
52 GROUP BY salesman_id;
```

<

Result Grid



Filter Rows:

Export:



Wrap Cell C

	salesman_id	total_sales
▶	5002	1349.45
	5001	11271.46
	5003	2590.90
	5005	270.65
	5006	1983.43
	5007	75.29

Result 11 ×

Output

```

53 • SELECT customer_id,
54         SUM(purch_amt) AS total_purchase
55 FROM orders
56 GROUP BY customer_id;

```

Result Grid |  Filter Rows: | Export: 

	customer_id	total_purchase
▶	3005	1099.00
	3002	8870.86
	3009	2590.90
	3007	2400.60
	3001	270.65
	3004	1983.43
	3003	75.29
	3008	250.45

```

57 • SELECT customer_id,
58         MAX(purch_amt) AS highest_purchase
59 FROM orders
60 GROUP BY customer_id;

```

Result Grid |  Filter Rows: | Export: 

	customer_id	highest_purchase
▶	3005	948.50
	3002	5760.00
	3009	2480.40
	3007	2400.60
	3001	270.65
	3004	1983.43
	3003	75.29
	3008	250.45

```
61 • SELECT salesman_id,  
62         SUM(purch_amt) AS total_sales  
63 FROM orders  
64 GROUP BY salesman_id  
65 HAVING SUM(purch_amt) > 3000;
```

<

Result Grid



Filter Rows:

Export:



	salesman_id	total_sales
▶	5001	11271.46

```
66 • SELECT customer_id,  
67         SUM(purch_amt) AS total_purchase  
68 FROM orders  
69 GROUP BY customer_id  
70 HAVING SUM(purch_amt) > 2500;
```

<

Result Grid



Filter Rows:

Export:



	customer_id	total_purchase
▶	3002	8870.86
	3009	2590.90

```

71 • SELECT customer_id,
72         COUNT(*) AS total_orders
73 FROM orders
74 GROUP BY customer_id
75 HAVING COUNT(*) > 1;

```

<

Result Grid   Filter Rows:

	customer_id	total_orders
▶	3005	2
	3002	3
	3009	2

```

76 • SELECT customer_id,
77         SUM(purch_amt) AS total_purchase
78 FROM orders
79 GROUP BY customer_id
80 HAVING SUM(purch_amt) > 1000
81 ORDER BY total_purchase DESC;

```

<

Result Grid   Filter Rows: Export: 

	customer_id	total_purchase
▶	3002	8870.86
	3009	2590.90
	3007	2400.60
	3004	1983.43
	3005	1099.00


```

82 • SELECT customer_id,
83         MAX(purch_amt) AS max_purchase
84 FROM orders
85 GROUP BY customer_id
86 HAVING MAX(purch_amt) BETWEEN 2000 AND 6000;

```

<

Result Grid   Filter Rows: Export:  Wrap

	customer_id	max_purchase
▶	3002	5760.00
	3009	2480.40
	3007	2400.60

```

87 • SELECT salesman_id,
88         COUNT(*) AS total_orders
89 FROM orders
90 GROUP BY salesman_id
91 HAVING COUNT(*) >= 2;

```

<

Result Grid   Filter Rows: |

	salesman_id	total_orders
▶	5002	3
	5001	4
	5003	2

```
92 • SELECT ord_date,  
93         MAX(purch_amt) AS highest_purchase  
94 FROM orders  
95 GROUP BY ord_date  
96 HAVING MAX(purch_amt) > 2000;
```

<

Result Grid



Filter Rows:

Export:



	ord_date	highest_purchase
▶	2012-10-10	2480.40
	2012-07-27	2400.60
	2012-09-10	5760.00
	2012-04-25	3045.60